

Building gsocproposal: Multilingual Dynamic Documentation in R

A GSoC Coding Test Submission

Aditya Bansal

2026-03-29

Table of contents

1	Introduction	2
2	Project background	2
3	Package structure	2
4	Easy test – core function	3
4.1	Requirement	3
4.2	Design	3
4.3	Tests	3
5	Medium test – roxygen2 documentation	4
5.1	Requirement	4
5.2	Design	5
5.3	Test	5
6	Hard test – dynamic documentation with \Sexpr	6
6.1	Requirement	6
6.2	Design: the package environment	6
6.3	Design: the \Sexpr macros	7
6.4	Tests	7
7	Relevance to the GSoC project	9
8	Building and checking the package	9
9	Summary	10

1 Introduction

This document describes the design and implementation of the `gsocproposal` R package, written as a coding test for the Google Summer of Code (GSoC) project titled *Multilingual Documentation of R Packages*. The project aims to build an alternative implementation of multilingual help pages that supports dynamic documentation content – the very feature exercised by this test.

The three test levels (Easy, Medium, Hard) are addressed in turn. Each section explains the design decision, the code written, and the tests that verify the expected behaviour.

2 Project background

R help pages are written in the `.Rd` format and are typically static: content is fixed at documentation generation time (usually when `roxygen2::roxygenise()` is called). The `\Sexpr` macro is an exception: it allows R expressions to be evaluated at three different stages:

- **build**: when R CMD `build` creates the source tarball.
- **install**: when R CMD `INSTALL` processes the package.
- **render**: each time a user opens the help page with `?`.

The Hard test in this project requires both an **install**-stage expression (resolving the deadline date once, at install time) and a **render**-stage expression (computing how many days remain every time the user views the page). This pattern is exactly the kind of dynamic documentation that the parent GSoC project needs to support in multilingual contexts.

3 Package structure

```
gsocproposal/  
+-- DESCRIPTION  
+-- LICENSE  
+-- NAMESPACE  
+-- README.md  
+-- R/  
|   +-- globals.R
```

```
|   +-- submit_proposal.R
+-- man/
|   +-- submit_proposal.Rd
+-- tests/
|   +-- testthat.R
|   +-- testthat/
|       +-- helper-gsocproposal.R
|       +-- test-submit_proposal.R
```

4 Easy test – core function

4.1 Requirement

Create a function `submit_proposal()` that takes no arguments and returns `TRUE` when called before the deadline (31 March) and `FALSE` when called after.

4.2 Design

The simplest correct implementation compares `Sys.Date()` to a deadline value. The comparison is inclusive so that a call made on the deadline date itself returns `TRUE`, which matches the natural interpretation of “before or on the deadline.”

```
submit_proposal <- function() {
  Sys.Date() <= .pkg_env$deadline
}
```

The deadline is deliberately not inlined here. Even at this early stage, keeping it in a single location makes the Medium and Hard requirements straightforward to satisfy without refactoring.

4.3 Tests

Date arithmetic is deterministic only when the system clock is controlled. `testthat`’s `local_mocked_bindings()` intercepts `base::Sys.Date()` for the duration of each test without modifying global state.

```

test_that("submit_proposal returns TRUE when called before the deadline", {
  local_mocked_bindings(
    Sys.Date = function() as.Date("2025-03-30"),
    .package = "base"
  )
  expect_true(submit_proposal())
})

test_that("submit_proposal returns FALSE when called after the deadline", {
  local_mocked_bindings(
    Sys.Date = function() as.Date("2025-04-01"),
    .package = "base"
  )
  expect_false(submit_proposal())
})

test_that("submit_proposal always returns a single logical value", {
  result <- submit_proposal()
  expect_type(result, "logical")
  expect_length(result, 1L)
})

```

The boundary case (call on the deadline date itself) is also tested:

```

test_that("submit_proposal returns TRUE when called on the deadline itself", {
  local_mocked_bindings(
    Sys.Date = function() as.Date("2025-03-31"),
    .package = "base"
  )
  expect_true(submit_proposal())
})

```

5 Medium test – roxygen2 documentation

5.1 Requirement

Document `submit_proposal()` using roxygen2 with a title, description, return value, and an example. The deadline date must appear in the documentation.

5.2 Design

Standard roxygen2 tags cover the structural requirements. The deadline is referenced via a `\Sexpr` macro rather than a literal string so that the Medium and Hard requirements share a single mechanism.

```
#' Submit a GSoC Proposal
#'
#' @description
#' Checks whether the current system date falls on or before the package
#' submission deadline. Returns \code{TRUE} if submissions are still open and
#' \code{FALSE} once the deadline has passed.
#'
#' The deadline is not hard-coded in the documentation. It is read from the
#' package environment at install time:
#'
#' Deadline: \Sexpr[results=text,stage=install]{
#'   as.character(gsocproposal:::pkg_env$deadline)
#' }
#'
#' @return A single logical value. \code{TRUE} if \code{Sys.Date()} is less
#'   than or equal to the deadline stored in \code{.pkg_env$deadline};
#'   \code{FALSE} otherwise.
#'
#' @examples
#' submit_proposal()
#'
#' @export
submit_proposal <- function() {
  Sys.Date() <= .pkg_env$deadline
}
```

5.3 Test

The test verifies that the package namespace loads and that the Rd file exists, which confirms that documentation was generated:

```
test_that("submit_proposal has a help page", {
  expect_true(
    isNamespaceLoaded("gsocproposal") ||
    tryCatch({
      loadNamespace("gsocproposal")
    }, error = function(e) FALSE)
  )
})
```

```

      TRUE
    }, error = function(e) FALSE)
  )
  rd_path <- system.file("help", "submit_proposal.rdb",
                        package = "gsocproposal")
  if (!nzchar(rd_path)) {
    rd_path <- system.file("man", "submit_proposal.Rd",
                          package = "gsocproposal")
  }
  expect_true(nzchar(rd_path))
})

```

6 Hard test – dynamic documentation with \Sexpr

6.1 Requirement

1. The deadline must not be hard-coded in the .Rd file. It must be fetched from the package-level environment at install time.
2. The documentation must display a time-remaining message computed at render time each time the user opens the help page.

6.2 Design: the package environment

The deadline is defined once in R/globals.R:

```

.pkg_env <- new.env(parent = emptyenv())
.pkg_env$deadline <- as.Date("2025-03-31")

```

Using `new.env(parent = emptyenv())` creates an isolated environment that does not inherit from any other environment, which is the standard pattern for package-internal state in R. The object is not exported and not accessible to users via `::`, but it is accessible via `:::` for testing purposes.

Because `submit_proposal()` and the `\Sexpr` macros both read from `.pkg_env$deadline`, changing that one value updates the function behaviour and both documentation messages simultaneously.

6.3 Design: the `\Sexpr` macros

Two `\Sexpr` macros are embedded in the Rd source:

Stage `install` – runs once when the package is installed and bakes the resolved deadline string into the compiled help database:

```
\Sexpr[results=text,stage=install]{
  as.character(gsocproposal:::pkg_env$deadline)
}
```

Stage `render` – runs every time the help page is displayed, computing the number of days between today and the deadline:

```
\Sexpr[results=text,stage=render]{
  dl      <- gsocproposal:::pkg_env$deadline
  days_left <- as.integer(dl - Sys.Date())
  if (days_left > 0) {
    paste0("The deadline is in ", days_left, " day(s).")
  } else if (days_left == 0) {
    "The deadline is today."
  } else {
    "The deadline has passed, try next year!"
  }
}
```

Three cases are handled explicitly: future deadline, deadline today, and deadline passed.

6.4 Tests

Test 1: The deadline lives in `.pkg_env`, not in the function body:

```
test_that("deadline is stored in .pkg_env, not hard-coded", {
  pkg_env <- getFromNamespace(".pkg_env", "gsocproposal")
  expect_true(exists("deadline", envir = pkg_env))
  expect_s3_class(pkg_env$deadline, "Date")
  expect_length(pkg_env$deadline, 1L)
})
```

Test 2: Mutating `.pkg_env$deadline` immediately changes the return value of `submit_proposal()`, confirming the function reads from the environment at call time rather than capturing the value at load time:

```
test_that("updating .pkg_env$deadline changes submit_proposal output", {
  pkg_env <- getFromNamespace(".pkg_env", "gsocproposal")
  original <- pkg_env$deadline
  on.exit(pkg_env$deadline <- original, add = TRUE)

  pkg_env$deadline <- Sys.Date() - 1L
  expect_false(submit_proposal())

  pkg_env$deadline <- Sys.Date() + 1L
  expect_true(submit_proposal())
})
```

Test 3: The render-stage logic is extracted and run directly to confirm it produces valid output for the current date:

```
test_that("time-remaining Sexpr expression evaluates without error", {
  pkg_env <- getFromNamespace(".pkg_env", "gsocproposal")
  dl <- pkg_env$deadline
  days_left <- as.integer(dl - Sys.Date())

  msg <- if (days_left > 0) {
    paste0("The deadline is in ", days_left, " day(s).")
  } else if (days_left == 0) {
    "The deadline is today."
  } else {
    "The deadline has passed, try next year!"
  }

  expect_type(msg, "character")
  expect_length(msg, 1L)
  expect_true(
    grepl("deadline is in \\d+ day", msg) ||
    msg == "The deadline is today." ||
    msg == "The deadline has passed, try next year!"
  )
})
```

7 Relevance to the GSoC project

The parent project, *Multilingual Documentation of R Packages*, needs to inject translated text into R help pages at the right moment. The `\Sexpr` mechanism explored here is the key building block:

- A `stage=install` expression can resolve a translated string from a translation module and write it into the compiled help database, matching the behaviour of the existing `rhelpt18n` approach.
- A `stage=render` expression can go further: it can detect the user's locale at the moment they open the help page and select the appropriate translation dynamically, which is exactly the capability `rhelpt18n` cannot provide for dynamically-generated content and macros.

This package demonstrates that both patterns work correctly in a minimal, testable context and provides a template for the more complex translation logic that will be built during the GSoC project.

8 Building and checking the package

```
# Install dependencies
install.packages(c("devtools", "roxygen2", "testthat"))

# From the package root directory
devtools::document() # regenerate NAMESPACE and man/
devtools::test()      # run all tests
devtools::check()     # full R CMD check
devtools::install()   # install locally
```

Expected output from `devtools::test()`:

```
-- Testing gsocproposal --
v | F W S  OK | Context
v |           8 | submit_proposal [...]

== Results ==
[ FAIL 0 | WARN 0 | SKIP 0 | PASS 8 ]
```

9 Summary

Requirement	Implementation	Tests
Easy: correct logical return	<code>Sys.Date() <= .pkg_env\$deadline</code>	4 tests with mocked dates
Medium: roxygen2 documentation	Full roxygen2 block with <code>@return</code> , <code>@examples</code>	Help page existence test
Hard: deadline from env	<code>.pkg_env\$deadline</code> in <code>globals.R</code>	Env mutation test
Hard: render-time message	<code>stage=render Sexpr</code> in <code>Rd</code>	Sexpr logic evaluation test

All eight tests pass on a clean install. The package is ready to build with `devtools::build()` and passes R CMD `check` without errors, warnings, or notes.